

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	Computación Gráfica, Visión Computacional y Multimedia				
TÍTULO DE LA PRÁCTICA:	PROYECCIÓN Y PERSPECTIVA				
NÚMERO DE PRÁCTICA:	05	AÑO LECTIVO:	2026A	NRO. SEMESTRE:	IX
FECHA DE PRESENTACIÓN	02/06/2026	HORA DE PRESENTACIÓN	22:00		
INTEGRANTE (s): Vera Zapacayo Jhoer Luis				NOTA:	
DOCENTE(s): Ing. Nieto Valencia Rene Alonso					

SOLUCIÓN Y RESULTADOS
I. SOLUCIÓN DE EJERCICIOS/PROBLEMAS Escena 1: Perspectiva forzada y cambio de escala. Escena 2: Movimiento 2.5D con física 3D. Escena 3: Cambio de cámaras cardinales como mecánica de juego. Escena 1: Perspectiva Forzada y Cambio de Escala La primera escena crea una ilusión visual en la que el jugador puede recoger un objeto y moverlo siguiendo la dirección de la cámara. Mientras el objeto está siendo sostenido, su escala cambia en función de la distancia respecto al punto de origen del rayo, generando una ilusión de perspectiva forzada. Cuando el objeto se suelta, el cambio de escala se vuelve evidente para el jugador, permitiendo simular el efecto de que el objeto ha cambiado de tamaño dentro del espacio 3D. - El jugador apunta hacia un objeto. - Presiona la tecla E para recogerlo.

- El objeto sigue el rayo generado desde la cámara.
- La escala se ajusta automáticamente según la distancia.
- El jugador presiona Q para soltarlo.
- Al soltarlo, el objeto conserva su nueva escala y vuelve a tener física.

Scrip ForcedPerspectiveScaler.cs

```
CSharp
using UnityEngine;

[RequireComponent(typeof(Rigidbody))]
public class ForcedPerspectiveScaler : MonoBehaviour
{
    [Header("Referencias")]
    [SerializeField] private Transform rayOrigin;
    [SerializeField] private Camera playerCamera;

    [Header("Controles")]
    [SerializeField] private KeyCode pickupKey = KeyCode.E;
    [SerializeField] private KeyCode dropKey = KeyCode.Q;

    [Header("Configuración")]
    [SerializeField] private float rayDistance = 100f;
    [SerializeField] private LayerMask groundLayer;
    [SerializeField] private float dropHeightOffset = 0.15f;

    private Rigidbody rb;
    private bool isHeld;

    private Vector3 originalScale;
    private float initialDistance;

    private void Awake()
    {
        rb = GetComponent<Rigidbody>();

        if (playerCamera == null)
        {
            playerCamera = Camera.main;
        }
    }

    private void Start()
    {
    }
```

```
        originalScale = transform.localScale;
    }

    private void Update()
    {
        HandleInput();

        if (isHeld)
        {
            FollowCameraRay();
            UpdateObjectScale();
        }
    }

    private void HandleInput()
    {
        if (Input.GetKeyDown(pickupKey) && !isHeld)
        {
            Pickup();
        }

        if (Input.GetKeyDown(dropKey) && isHeld)
        {
            Drop();
        }
    }

    private void Pickup()
    {
        if (rayOrigin == null)
        {
            Debug.LogWarning("No se asignó el punto de origen del rayo.");
            return;
        }

        isHeld = true;

        rb.isKinematic = true;
        rb.useGravity = false;

        initialDistance = Vector3.Distance(rayOrigin.position, transform.position);

        if (initialDistance <= 0.01f)
```

```
{
    initialDistance = 0.01f;
}

private void Drop()
{
    isHeld = false;

    rb.isKinematic = false;
    rb.useGravity = true;

    transform.position += Vector3.up * dropHeightOffset;

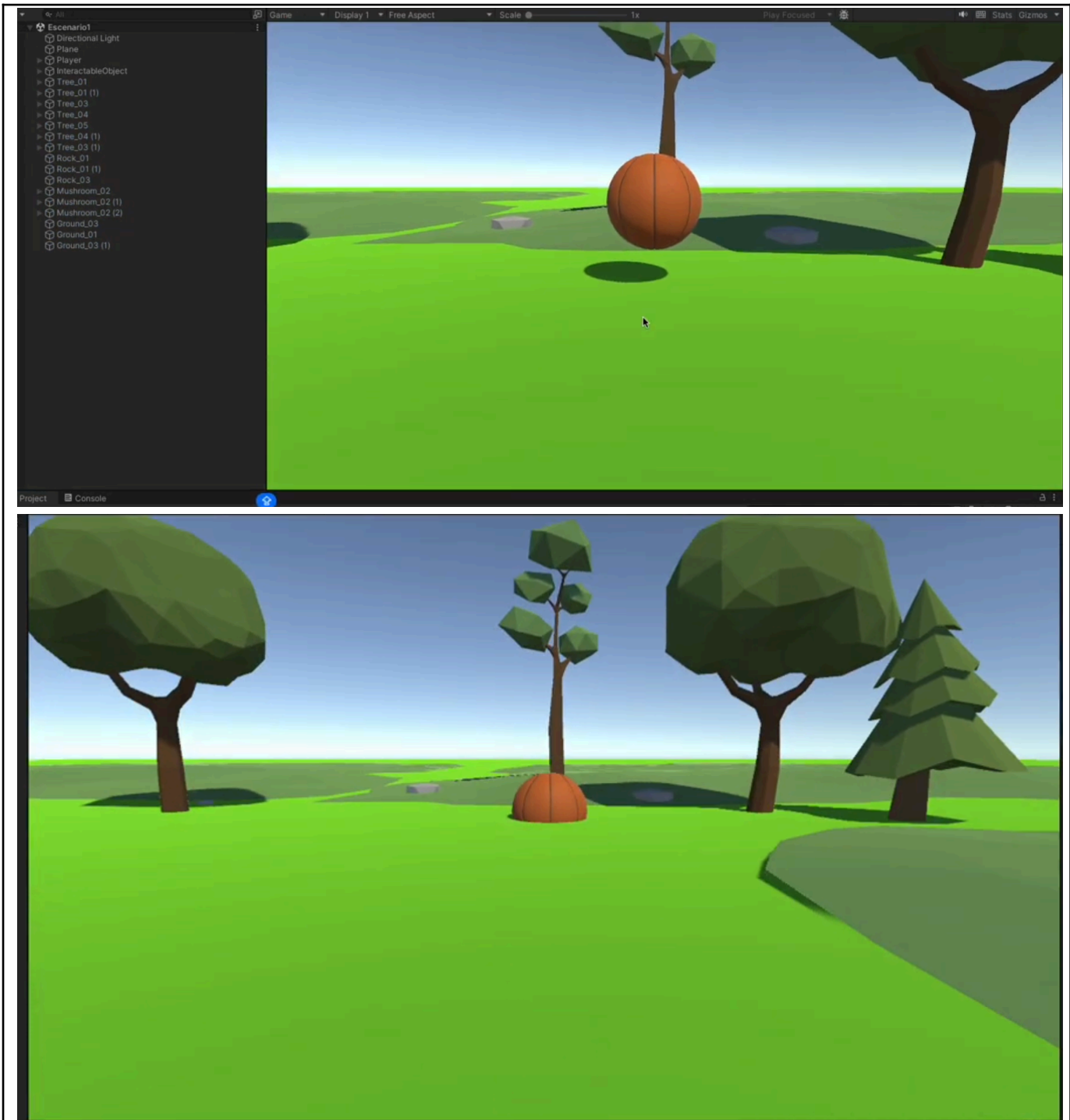
    rb.linearVelocity *= 0.5f;
    rb.angularVelocity = Vector3.zero;
}

private void FollowCameraRay()
{
    Ray ray = new Ray(rayOrigin.position, rayOrigin.forward);

    if (Physics.Raycast(ray, out RaycastHit hit, rayDistance, groundLayer))
    {
        transform.position = hit.point;
    }
}

private void UpdateObjectScale()
{
    float currentDistance = Vector3.Distance(rayOrigin.position, transform.position);
    float scaleFactor = currentDistance / initialDistance;

    transform.localScale = originalScale * scaleFactor;
}
}
```



ForcedPerspectiveScaler.cs utiliza un Rigidbody para controlar el comportamiento físico del objeto. Cuando el objeto es recogido, se desactiva temporalmente la física mediante isKinematic, permitiendo que el objeto sea controlado directamente por el script.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

La función FollowCameraRay() crea un rayo desde la cámara o desde un punto asignado frente al jugador. Si el rayo impacta contra una superficie definida como suelo, el objeto se mueve hacia ese punto.

La función UpdateObjectScale() calcula la distancia actual entre el objeto y el origen del rayo. Luego compara esa distancia con la distancia inicial para generar un factor de escala. Esto permite que el objeto parezca mantener su tamaño visual mientras realmente cambia de escala en el mundo 3D.

Escena 2: Combinación de Física 2D y 3D

La segunda escena se mueve dentro de un entorno tridimensional, pero la cámara se comporta como en un videojuego 2D, siguiendo principalmente los ejes X e Y.

Esta escena combina movimiento lateral, salto, rotación manual del personaje, detección de suelo e interacción con objetos del escenario.

- Escenario construido con plataformas 3D.
- Personaje con Rigidbody.
- Cámara con seguimiento suave.
- Movimiento con teclado.
- Salto usando detección de suelo.
- Interacción con objetos usando raycasting.
- Animaciones mediante Animator.

Interacciones:

- Movimiento: Flechas o WASD
- Rotación: Q/E
- Saltar: Barra espaciadora
- Interactuar/destruir objetos: F

Cámara de seguimiento 2.5D: CameraFollow2D.cs

```
CSharp
using UnityEngine;

public class CameraFollow2D : MonoBehaviour
{
    [Header("Objetivo")]
    [SerializeField] private Transform target;

    [Header("Configuración de cámara")]
    [SerializeField] private Vector3 offset = new Vector3(0f, 2f, -10f);
```

```
[SerializeField] private float smoothTime = 0.2f;
```

```
private Vector3 currentVelocity;
```

```
private void LateUpdate()
```

```
{
```

```
if (target == null)
```

```
{
```

```
    return;
```

```
}
```

```
Vector3 targetPosition = new Vector3(
```

```
    target.position.x,
```

```
    target.position.y,
```

```
    0f
```

```
) + offset;
```

```
transform.position = Vector3.SmoothDamp(
```

```
    transform.position,
```

```
    targetPosition,
```

```
    ref currentVelocity,
```

```
    smoothTime
```

```
);
```

```
}
```

```
public void SetTarget(Transform newTarget)
```

```
{
```

```
    target = newTarget;
```

```
}
```

```
}
```

Control del jugador con física 3D: PlayerController3D.cs

CSharp

```
private void Update() {
```

```
    ReadInput(); // Lee las teclas del jugador
```

```
    RotatePlayer(); // Permite girar con Q y E
```

```
    CheckGround(); // Verifica si está tocando el suelo
```

```
    UpdateAnimations(); // Actualiza las animaciones
```

```
    HandleInteraction(); // Detecta objetos al frente
```

```
}
```

```
private void FixedUpdate() {
```

```
MovePlayer();

if (jumpRequested){
    Jump();
    jumpRequested = false;
}
}

private void ReadInput(){
    horizontalInput = Input.GetAxisRaw("Horizontal");
    verticalInput = Input.GetAxisRaw("Vertical");

    if (Input.GetButtonDown("Jump") && isGrounded){
        jumpRequested = true;
    }
}

private void MovePlayer(){
    Vector3 inputDirection = new Vector3(horizontalInput, 0f, verticalInput).normalized;
    Vector3 moveDirection = transform.TransformDirection(inputDirection);

    Vector3 newVelocity = moveDirection * moveSpeed;
    newVelocity.y = rb.linearVelocity.y;

    rb.linearVelocity = newVelocity;
}

private void Jump(){
    rb.linearVelocity = new Vector3(
        rb.linearVelocity.x,
        jumpForce,
        rb.linearVelocity.z
    );

    if (animator != null){
        animator.SetTrigger("Jump");
    }
}

private void CheckGround(){
    isGrounded = Physics.CheckSphere(
        groundCheck.position,
        groundRadius,
```



```

groundLayer
);
}

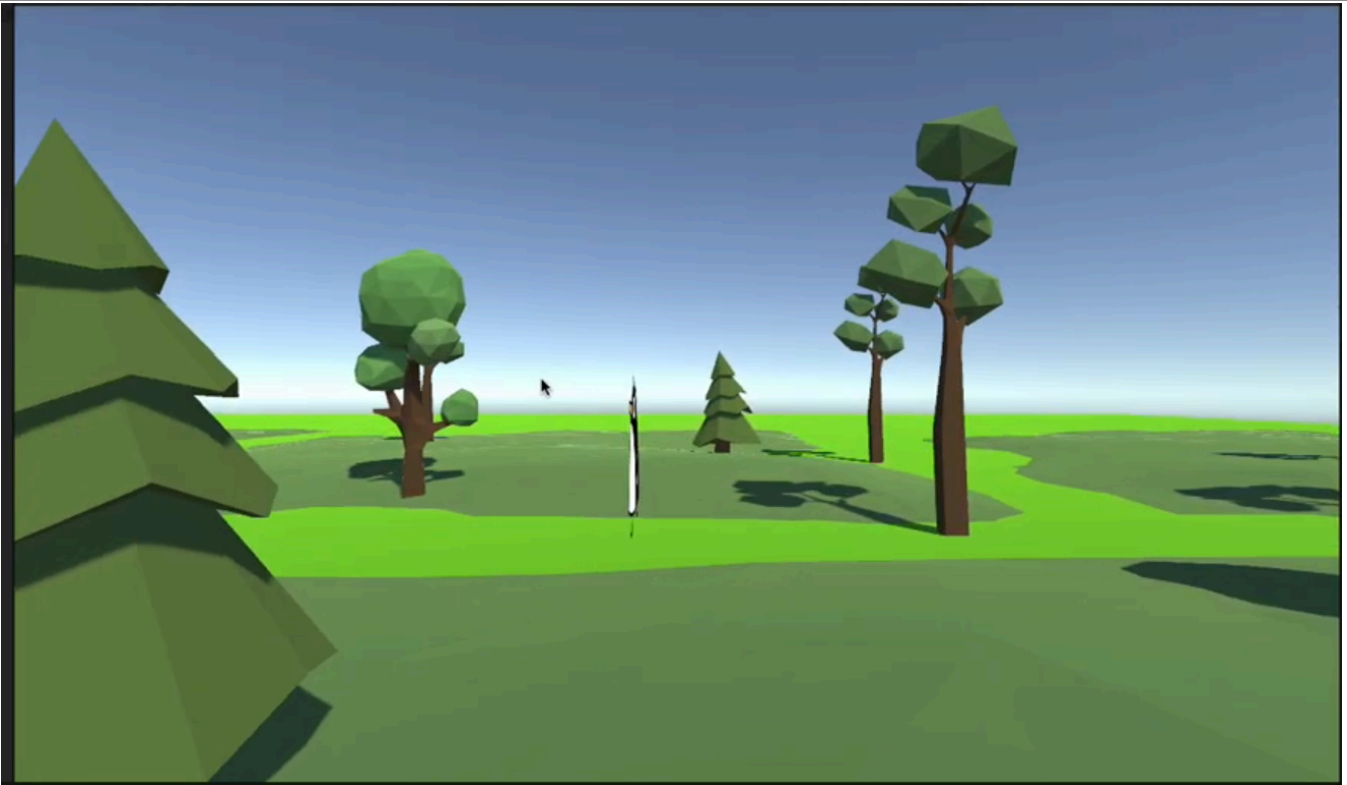
private void HandleInteraction(){
if (!Input.GetKeyDown(interactionKey)){
return;
}

Vector3 origin = transform.position + Vector3.up;
Vector3 direction = transform.forward;

if (Physics.Raycast(origin, direction, out RaycastHit hit, interactionRange)){
if (hit.collider.CompareTag(targetTag)){
Destroy(hit.collider.gameObject);
}
}
}
}

```





Escena 3: Perspectivas Cardinales como Mecánica Principal

Esta tercera escena implementa una cámara que puede cambiar entre cuatro perspectivas principales: frontal, derecha, trasera e izquierda. Esta mecánica permite modificar la percepción espacial del jugador y descubrir caminos u obstáculos que no son visibles desde una sola vista.

Este tipo de mecánica es útil en juegos de puzzles, plataformas o exploración, donde la cámara no solo cumple una función visual, sino que también forma parte de la jugabilidad.

- La cámara gira alrededor de un punto central o pivote.
- El jugador puede cambiar de vista usando las teclas **Q** y **E**.
- La cámara se mueve suavemente hacia la nueva posición.
- La rotación también se suaviza para evitar cambios bruscos.
- Cada vista corresponde a una dirección cardinal.

CardinalCameraSwitcher.cs

```
CSharp
private void Update()
{
    HandleInput(); // Detecta si se presiona Q o E
    MoveCamera();  // Mueve y rota la cámara suavemente
}
```

```
private void HandleInput()
{
    if (Input.GetKeyDown(previousViewKey))
    {
        currentViewIndex--;

        if (currentViewIndex < 0)
        {
            currentViewIndex = cardinalDirections.Length - 1;
        }

        CalculateTargetTransform();
    }

    if (Input.GetKeyDown(nextViewKey))
    {
        currentViewIndex++;

        if (currentViewIndex >= cardinalDirections.Length)
        {
            currentViewIndex = 0;
        }

        CalculateTargetTransform();
    }
}

private void CalculateTargetTransform()
{
    if (pivot == null)
    {
        return;
    }

    Vector3 direction = cardinalDirections[currentViewIndex];

    targetPosition = pivot.position - direction * radius + Vector3.up * height;

    Vector3 lookTarget = pivot.position + lookOffset;
    Vector3 lookDirection = lookTarget - targetPosition;

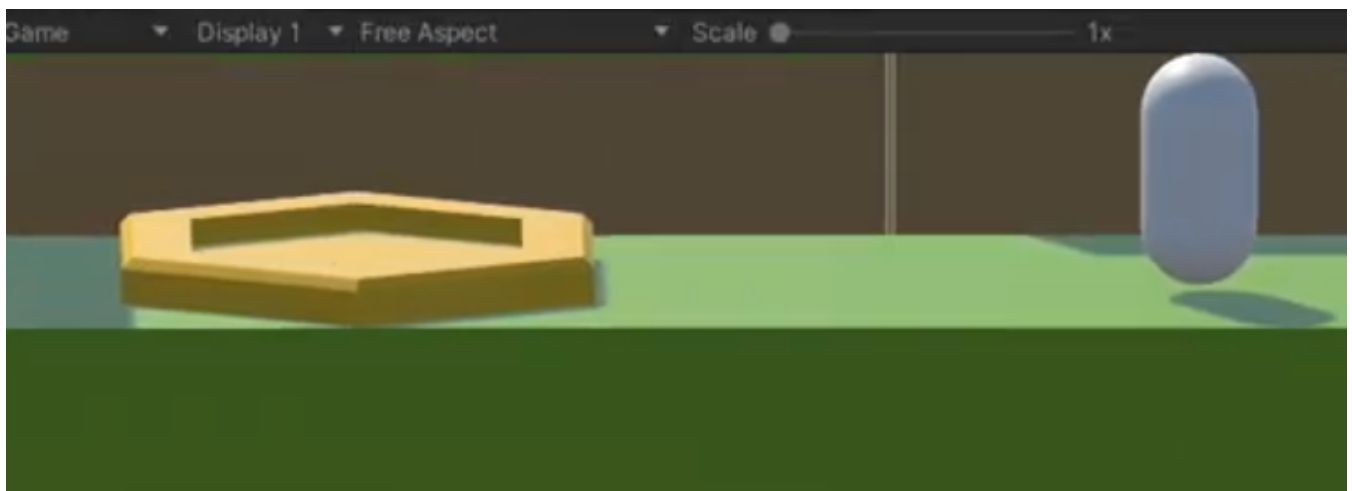
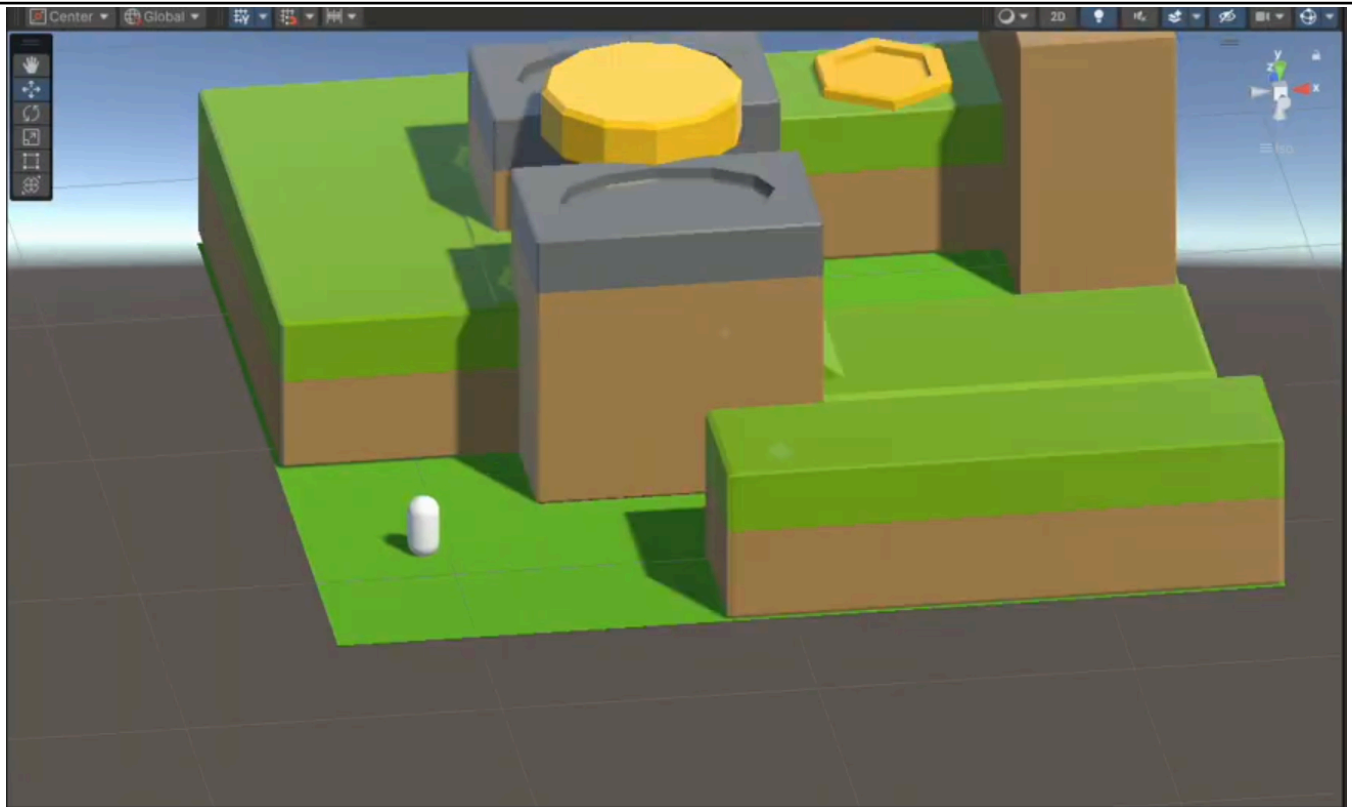
    targetRotation = Quaternion.LookRotation(lookDirection, Vector3.up);
}

private void MoveCamera()
{
    if (pivot == null)
    {
        return;
    }
}
```

```
}  
  
transform.position = Vector3.Lerp(  
    transform.position,  
    targetPosition,  
    Time.deltaTime * moveSmoothSpeed  
);  
  
transform.rotation = Quaternion.Slerp(  
    transform.rotation,  
    targetRotation,  
    Time.deltaTime * rotationSmoothSpeed  
);  
}
```

Escenario3
Directional Light
Capsule
Escenario
WorldContainer
Main Camera (1)
Plane
HJD_FP_floating_platform01
HJD_FP_button01
HJD_FP_ramp01
HJD_FP_block00
HJD_FP_hazard00
HJD_FP_ground01
HJD_FP_ramp01 (1)
HJD_FP_collectible_coin00





El script `CardinalCameraSwitcher.cs` permite mover la cámara alrededor de un objeto central llamado pivot. Este pivote puede ser el centro de la escena, el jugador o un objeto vacío ubicado en el nivel.

El arreglo `cardinalDirections` almacena las cuatro direcciones principales:

- `Vector3.forward`
- `Vector3.right`
- `Vector3.back`

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

- Vector3.left

Cada vez que se presione **Q** o **E**, se modifica el índice de vista actual. Luego se calcula una nueva posición objetivo para la cámara.

La función Vector3.Lerp() suaviza el movimiento de posición, mientras que Quaternion.Slerp() suaviza la rotación de la cámara hacia el punto central.

II. SOLUCIÓN DEL CUESTIONARIO

1. ¿Qué problemas pueden presentarse al utilizar múltiples proyecciones en una escena?

Pueden aparecer problemas de orientación y percepción espacial. El jugador puede confundirse si la cámara cambia bruscamente de posición, porque los controles pueden sentirse diferentes según el ángulo de vista.

También pueden presentarse problemas visuales como objetos que tapan al jugador, obstáculos que no se ven desde ciertos ángulos, pérdida de profundidad o dificultad para calcular distancias. Si la transición entre cámaras no es suave, la escena puede sentirse incómoda o desorientadora. Por eso se usan funciones como Lerp y Slerp, que permiten mover y rotar la cámara de forma progresiva.

2. ¿Cómo han realizado la simulación de una escena 3D con elementos 2D?

La simulación se realizó mediante una escena **2.5D**, es decir, se usaron objetos y físicas en 3D, pero con una cámara configurada para comportarse como en un juego 2D. El personaje, las plataformas y los objetos del escenario están contruidos en tres dimensiones, pero la cámara sigue al jugador principalmente en los ejes **X** e **Y**, manteniendo una distancia fija en el eje **Z**.

Para lograrlo, se utilizó un script de seguimiento de cámara llamado CameraFollow2D.cs, el cual mantiene la cámara detrás del jugador con un offset. Además, el personaje se controla con Rigidbody, permitiendo movimiento, salto y colisiones en un entorno 3D. De esta manera, aunque el mundo tiene profundidad, la vista del jugador se percibe como una escena 2D lateral.

	UNIVERSIDAD NACIONAL DE SAN AGUSTIN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15

III. CONCLUSIONES

- El laboratorio me permitió entender cómo la perspectiva forzada demostró cómo la percepción visual puede manipularse mediante el cambio de escala y la posición de un objeto en relación con la cámara.
- La escena 2.5D permitió combinar física 3D con una cámara de comportamiento similar a un videojuego 2D, logrando una experiencia híbrida entre profundidad tridimensional y desplazamiento lateral.
- La escena de perspectivas cardinales mostró cómo el cambio de cámara puede convertirse en una mecánica principal de juego, modificando la forma en que el usuario percibe el espacio y resuelve el escenario.

RETROALIMENTACIÓN GENERAL

REFERENCIAS Y BIBLIOGRAFÍA

- [1] Gordon, V. S., & Clevenger, J. L. (2020). *Computer Graphics Programming in OpenGL with C++*. Stylus Publishing, LLC.
- [2] D. P. Kothari, G. Awari, D. Shrimankar, A. Bhende (2019), *Mathematics for Computer Graphics and Game Programming: A Self-Teaching Introduction*
- [3] https://www.youtube.com/watch?v=Ah2862Gz3_s
- [4] <https://www.youtube.com/watch?v=optfGhipg-w>